Introduction to Software Engineering

Software Design



Software Engineering Department
Faculty of Information and Technology
University of Science

Contents

Objectives 4

1 Member Contribution Assessment 4

2 Design Overview 5

2.1 Design Goals and Constraints 5

3 Architectural Design 9

3.1 Architectural Style 9

3.2 System Context Diagram (C4 level 1) 10

3.3 Container Diagram (C4 level 2) 10

3.4 Component Diagram 10

3.5 Deployment Diagram 10

3.6 Architecture Decision Records 10

4 Object-Oriented Design 11

4.1 Domain Model 11

4.2 Class Diagram 16

4.3 Class Specifications 16

4.4 Sequence Diagrams 17

4.5 State Machine Diagrams 17

4.6 Design Principles and Patterns 17

5 Data Design 17

5.1 Data Diagram 17

5.2 Data Specification 17

5.3 Normalisation and Indexes 18

6 User Interface and User Experience Design 18

6.1 Screen List and Screen Flow 18
6.2 Screen Specifications 18
6.3 Accessibility Checklist 19

7 Traceability Update **19**

8 Changes to the Requirements Since PA2 **19**

9 AI Usage Log **20**

Software Design

Objectives

This document focus on the following topics:

- ✓ Completing the Software Design document with the following sections:
  - Architectural design (C4, ADR, deployment)
  - Object-oriented design (domain model, class and sequence diagrams)
  - Data design
  - User interface design
  - Traceability update
- ✓ Including an AI usage log for this phase
- ✓ Understanding the Software Design document.

1 Member Contribution Assessment

The contribution percentages and assigned design modules for the Software Design phase (PA3) were agreed by the team. They total 100%. Evidence is taken from the team's GitHub issues, commits, and merged pull requests.

ID	Name	Role(s)	Contribution (%)	Evidence
24127434	Đỗ Vĩnh Kỳ	Team Lead / PM	20	Overall design integration, architectural style, and C4 models review; repository commits.

24127374	Huỳnh Thái Hòa	Business Analyst	20	Section 2.1 (Design Goals, Priority FRs/NFRs, Constraints), Section 4.1 (Domain Model, TikZ diagram, Business Rules Verification), and Handover documentation; branch feature/pa3-sections-2.1-4.1.
24127117	Nguyễn Anh Tá	Designer / Architect	20	Sections 4.2--4.6 (Class, Sequence, State Machine diagrams) and Section 6 (UI/UX Design).
24127357	Nguyễn Gia	Developer	20	Section 3 (ADRs, Deployment diagram) and Section 5 (Data Design, Schema, Normalisation).
24127490	Trần Thiên Phú	Tester	20	Section 7 (Traceability Update) and Section 8 (Changes to Requirements since PA2).

2 Design Overview

2.1 Design Goals and Constraints

Thiết kế kiến trúc và mô hình hóa hệ thống US-Recovery (CampusHub) được xây dựng dựa trên việc thỏa mãn các Yêu cầu có ý nghĩa kiến trúc (Architecturally Significant Requirements -- ASRs). Các yêu cầu này bao gồm các tính năng nghiệp vụ cốt lõi (Core Functional Drivers), các thuộc tính chất lượng then chốt (Key Quality Attributes / NFRs), cùng hệ thống các ràng buộc kỹ thuật, ngân sách và tiến độ khắt khe của dự án môn học Nhập môn Công nghệ Phần mềm.

2.1.1 Architecturally Significant Functional Requirements

Các yêu cầu chức năng có mức ưu tiên cao nhất (Must-have) được xác định từ PA1 và đặc tả chi tiết trong PA2 đóng vai trò then chốt trong việc định hình các thành phần kiến trúc, mô hình dữ liệu và cơ chế bảo mật của hệ thống:

ID	Tính năng (Feature)	Yêu cầu chức năng cốt lõi	Tác động định hướng thiết kế

FR-01	Xác thực tài khoản (@...edu.vn)	Bắt buộc đăng ký và đăng nhập thông qua email chính danh trường đại học. Cung cấp mã OTP/link kích hoạt để định danh sinh viên.	Thiết kế module xác thực (Auth Service) tích hợp Supabase Auth và SMTP Service; phát hành JWT token để xác thực phiên làm việc không trạng thái (stateless).
FR-02	Phân quyền người dùng (RBAC)	Phân định rạch ròi 3 vai trò: Khách (Guest -- chỉ xem/tìm kiếm), Sinh viên (Student -- đăng bài, claim, quản lý tin) và Quản trị viên (Admin -- kiểm duyệt, cấu hình).	Cài đặt Middleware kiểm soát truy cập phân tầng (Role-based Authorization Middleware) tại Backend và cơ chế Route Guards bảo vệ các trang điều hướng tại Frontend.
FR-03	Quản lý bài đăng Mất/Nhặt đồ	Cho phép sinh viên tạo, sửa, đóng bài đăng; đính kèm ảnh thực tế, chọn cơ sở trường, phân loại danh mục, thời gian và địa điểm chi tiết.	Thiết kế mô hình dữ liệu tách biệt giữa CSDL quan hệ (PostgreSQL lưu metadata) và Cloud Storage (lưu trữ tệp ảnh tĩnh); xây dựng máy trạng thái bài đăng (Active, Resolved, Closed).
FR-04	Tìm kiếm & Lọc đa chiều	Cho phép tìm kiếm toàn văn theo từ khóa và bộ lọc kết hợp: Cơ sở trường, Danh mục vật phẩm, Khoảng thời gian, Loại bài đăng (Mất/Nhặt).	Thiết kế chỉ mục tìm kiếm tối ưu (PostgreSQL B-Tree trên các trường lọc, GIN index trên tiêu đề/nội dung), xử lý phân trang dữ liệu (Pagination) giảm tải mạng.
FR-05 FR-06	Quy trình nhận đồ (Claim Workflow)	Người mất gửi yêu cầu Claim kèm câu trả lời xác minh hoặc ảnh bằng chứng. Người nhặt duyệt (Accept/Reject) để mở khóa thông tin liên hệ bảo mật.	Thiết kế luồng xử lý trạng thái Claim (State Pattern), cơ chế kiểm duyệt hai bước (Two-step verification), và che giấu dữ liệu nhạy cảm (Data Masking) tại tầng API.
FR-07 FR-08	Kiểm duyệt & Quản trị nền tảng	Bảng điều khiển Admin xử lý báo cáo bài đăng vi phạm (ngưỡng $\geq$ 3 báo cáo theo BR-03 sẽ tự động chuyển vào hàng đợi), khóa tài khoản, duyệt cơ sở mới.	Thiết kế kiến trúc hàng đợi xử lý báo cáo (Moderation Queue), trigger cập nhật trạng thái tự động và các endpoint quản trị chuyên biệt được bảo vệ nghiêm ngặt.
FR-09	Thông báo nội bộ (In-app Notification)	Tự động gửi thông báo cho người dùng khi có tương tác Claim mới, thay đổi trạng thái bài đăng hoặc cảnh báo từ quản trị viên.	Thiết kế phân hệ lưu trữ và phân phối thông báo (Notification Subsystem), hỗ trợ truy vấn danh sách thông báo chưa đọc theo thời gian thực.

2.1.2 Key Quality Attributes and Non-Functional Requirements

Hệ thống phải đáp ứng các tiêu chuẩn chất lượng (NFRs) đã cam kết trong PA2, qua đó chi phối trực tiếp đến các giải pháp kiến trúc:

ID	Thuộc tính (Quality)	Chỉ số đo lường (Measurable Metric)	Giải pháp thiết kế kiến trúc
NFR-01	Performance (Hiệu năng)	Dưới tải 100 người dùng đồng thời, thời gian phản hồi tìm kiếm/lọc phải đạt P95 $\leq$ 2 giây.	Thiết kế kiến trúc phi trạng thái (Stateless REST API); thiết lập chỉ mục (Indexes) trên các khóa ngoại và cột lọc; giới hạn kích thước phân trang (Page Size); nén ảnh phía client trước khi upload.
NFR-02	Security & Privacy (Bảo mật & Riêng tư)	100% kết nối sử dụng giao thức HTTPS. Tuyệt đối không để lộ thông tin liên hệ (SĐT, Email cá nhân) của chủ bài đăng cho tới khi Claim được chấp thuận (BR-02).	Triển khai cơ chế Data Masking tại Business Service; áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege); cấu hình Row Level Security (RLS) trên PostgreSQL; băm mật khẩu bằng thuật toán bcrypt.
NFR-03	Usability (Tính khả dụng)	Sinh viên mới có thể hoàn tất việc tạo một bài đăng hoàn chỉnh trong vòng $\leq$ 3 phút mà không cần tài liệu hướng dẫn.	Thiết kế luồng giao diện tinh gọn, trực quan (Wizard/Modal form); tự động điền thông tin người dùng từ token; cung cấp phản hồi lỗi trực quan ngay tại trường nhập liệu (Client-side validation).
NFR-04	Compatibility (Khả năng tương thích)	Giao diện thích ứng (Responsive) hoàn hảo trên các trình duyệt Chrome, Firefox, Safari từ màn hình di động (360px) đến máy tính để bàn (1920px), không bị vỡ bố cục hoặc cuộn ngang.	Sử dụng kiến trúc Single Page Application (SPA) với React và Tailwind CSS theo triết lý Mobile-First; xây dựng các UI component linh hoạt (Flexbox, CSS Grid).
NFR-05	Availability (Tính sẵn sàng)	Hệ thống duy trì tính sẵn sàng (uptime) $\geq$ 99% trong suốt giai đoạn thử nghiệm và đánh giá nghiệm thu.	Tận dụng hạ tầng đám mây PaaS/BaaS (Vercel, Render) có cơ chế auto-restart khi gặp sự cố; thiết lập giám sát định kỳ thông qua UptimeRobot; tách biệt database độc lập với application server.

2.1.3 Verification of Core Business Rules

Hệ thống thiết kế phải đảm bảo thực thi triệt để 4 quy tắc nghiệp vụ cốt lõi đã cam kết từ PA2, đóng vai trò là các chốt chặn kiến trúc (Architectural Invariants):

Quy tắc	Nội dung quy tắc	Cơ chế kiến trúc bảo đảm	Tiêu chí xác minh (Verification)
BR-01	Xác thực Email trường (@...edu.vn)	Chỉ những tài khoản đã xác minh email chính danh trường đại học mới được phép tạo bài đăng và gửi Claim.	Auth Middleware kiểm tra cờ `is_verified = true` trước khi cho phép gửi request tạo Post hoặc Claim. Khách chỉ có quyền đọc/tìm kiếm (Read-only).
BR-02	Bảo vệ thông tin liên hệ (Protected Contact)	Số điện thoại và email cá nhân của chủ bài đăng tuyệt đối không hiển thị công khai; chỉ được mở khóa khi Claim được chấp thuận.	Cơ chế Data Masking tại tầng Business Service. API chi tiết bài đăng mặc định loại bỏ trường thông tin liên hệ trừ khi caller là chủ sở hữu hoặc Claimant đã được duyệt.
BR-03	Ngưỡng kiểm duyệt (Moderation Threshold)	Bất kỳ bài đăng nào nhận từ 3 báo cáo vi phạm trở lên ($\geq$ 3 reports) sẽ tự động bị ẩn khỏi danh sách công khai và chuyển vào hàng đợi Admin.	Database Trigger / Service Hook đếm số lượng bản ghi Report hợp lệ. Khi vượt ngưỡng 3, tự động chuyển `status` của Post sang `REPORTED` và gửi thông báo cảnh báo.
BR-04	Quy trình duyệt Claim (Claim Approval)	Người nhặt có toàn quyền đánh giá câu trả lời bí mật / bằng chứng ảnh để duyệt (Accept) hoặc từ chối (Reject) Claim. Khi duyệt, bài đăng tự động hoàn tất.	Thiết kế máy trạng thái Claim hữu hạn (`PENDING` $\rightarrow$ `ACCEPTED` / `REJECTED`). Sử dụng Database Transaction để đồng thời cập nhật Claim sang `ACCEPTED` và Post sang `RESOLVED`.

2.1.4 Design and Implementation Constraints

Thiết kế phần mềm phải tuân thủ nghiêm ngặt các giới hạn về công nghệ, tài chính và tiến độ:

1. **Ràng buộc Công nghệ (Technology Constraints):**
   - **Frontend:** Ứng dụng nền tảng web React 18+ kết hợp Vite làm công cụ build tối ưu, sử dụng React Router cho điều hướng SPA và Tailwind CSS để xây dựng hệ thống design tokens responsive.
   - **Backend:** Sử dụng Node.js (LTS) với nền tảng Express.js để xây dựng RESTful Web APIs chuẩn JSON. Cấu trúc mã nguồn tuân thủ nghiêm ngặt kiến trúc phân tầng (Layered Architecture: Routing -- Controller -- Service -- Data Access).

- **Cơ sở dữ liệu & Backend Services:** Sử dụng dịch vụ Supabase trên nền PostgreSQL 15+, tận dụng cơ chế Supabase Auth cho quản lý phiên và Supabase Storage làm kho lưu trữ hình ảnh vật phẩm.
- **Dịch vụ bên thứ ba:** Tích hợp Brevo SMTP Server để gửi email kích hoạt và mã xác thực OTP chính danh sinh viên.
- **Triển khai (Deployment):** Mã nguồn Frontend triển khai tự động trên Vercel CDN; Backend triển khai dưới dạng web service trên nền tảng Render (máy chủ đặt tại khu vực Singapore để tối ưu độ trễ mạng đối với người dùng tại TP.HCM).

2. **Ràng buộc Ngân sách (Budget Constraints):**
   - **Chi phí triển khai 0 VNĐ (Zero-Budget):** Dự án vận hành hoàn toàn dựa trên các gói tài nguyên miễn phí (Free Tiers) của các nền tảng đám mây (Vercel Hobby, Render Free Web Service, Supabase Free Project, Brevo Free 300 emails/day).
   - **Tối ưu hóa tài nguyên phần cứng:** Do gói máy chủ Render Free bị giới hạn RAM ở mức 512MB và có hiện tượng sleep sau thời gian nhàn rỗi, kiến trúc Backend phải thiết kế gọn nhẹ (lightweight), không chạy các tác vụ chạy nền (background workers) tốn bộ nhớ, đồng thời phía Frontend cần có thông báo chờ tải (skeleton loading/spinner) thân thiện.
3. **Ràng buộc Tiến độ và Quy trình (Schedule and Process Constraints):**
   - **Phương pháp luận phát triển:** Tuân thủ mô hình Waterfall theo đúng đề cương môn học, chia làm 5 mốc nghiệm thu tương ứng với các đồ án thành phần (PA1 đến PA5). Thiết kế tại PA3 đóng vai trò là bản thiết kế khung (Blueprint) bất biến cho giai đoạn cài đặt.
   - **Thời hạn lập trình (Coding Window):** Giai đoạn hiện thực mã nguồn (PA5) bị giới hạn nghiêm ngặt trong tối đa **3 tuần** làm việc của nhóm 5 thành viên. Do đó, thiết kế hệ thống phải tập trung cao độ vào tính khả thi, hạn chế thiết kế dư thừa (over-engineering), ưu tiên hoàn thiện xuất sắc 100% các tính năng cốt lõi (Must-have).
   - **Quy chuẩn làm việc nhóm:** Quản lý mã nguồn tập trung trên GitHub; 100% commit phải tuân thủ chuẩn Conventional Commits; bắt buộc tạo Pull Request và trải qua quy trình Peer Review (ít nhất 1 thành viên kiểm tra chéo) trước khi tích hợp vào nhánh chính (`main`).

3 Architectural Design

3.1 Architectural Style

[Name the architectural style(s) used (layered, MVC, client--server, microservices, event-driven, ...) and justify the choice against the design goals.]

3.2 System Context Diagram (C4 level 1)

[Update the context diagram from PA1.]

3.3 Container Diagram (C4 level 2)

[Show web/mobile apps, API/back end, database, external services and the technology of each container.]

3.4 Component Diagram

[Decompose the main container(s) into components/modules and show their dependencies (system decomposition).]

3.5 Deployment Diagram

[Show where each component runs (devices, servers/cloud services, database server) and how they communicate.]

3.6 Architecture Decision Records

[Record at least 3 important decisions (e.g., database, framework, authentication, hosting). One table per ADR; store them in `/docs/design/adr`.]

ADR	ADR-001: <title>
Status	proposed | accepted | superseded
Context	
Decision	
Alternatives considered	

Consequences	

4 Object-Oriented Design

4.1 Domain Model

Mô hình lĩnh vực (Domain Model) biểu diễn các thực thể khái niệm (Conceptual Business Entities) và mối quan hệ ngữ nghĩa thế giới thực trong hệ thống US-Recovery. Tuân thủ nguyên lý phân tích hướng đối tượng (OOA), Domain Model tập trung vào việc làm rõ bản chất nghiệp vụ, ý nghĩa thuộc tính và các quy tắc bất biến, **hoàn toàn không chứa phương thức xử lý (no methods)** cũng như không phụ thuộc vào chi tiết cài đặt kỹ thuật (khóa ngoại, chỉ mục, REST controller hay giao diện).

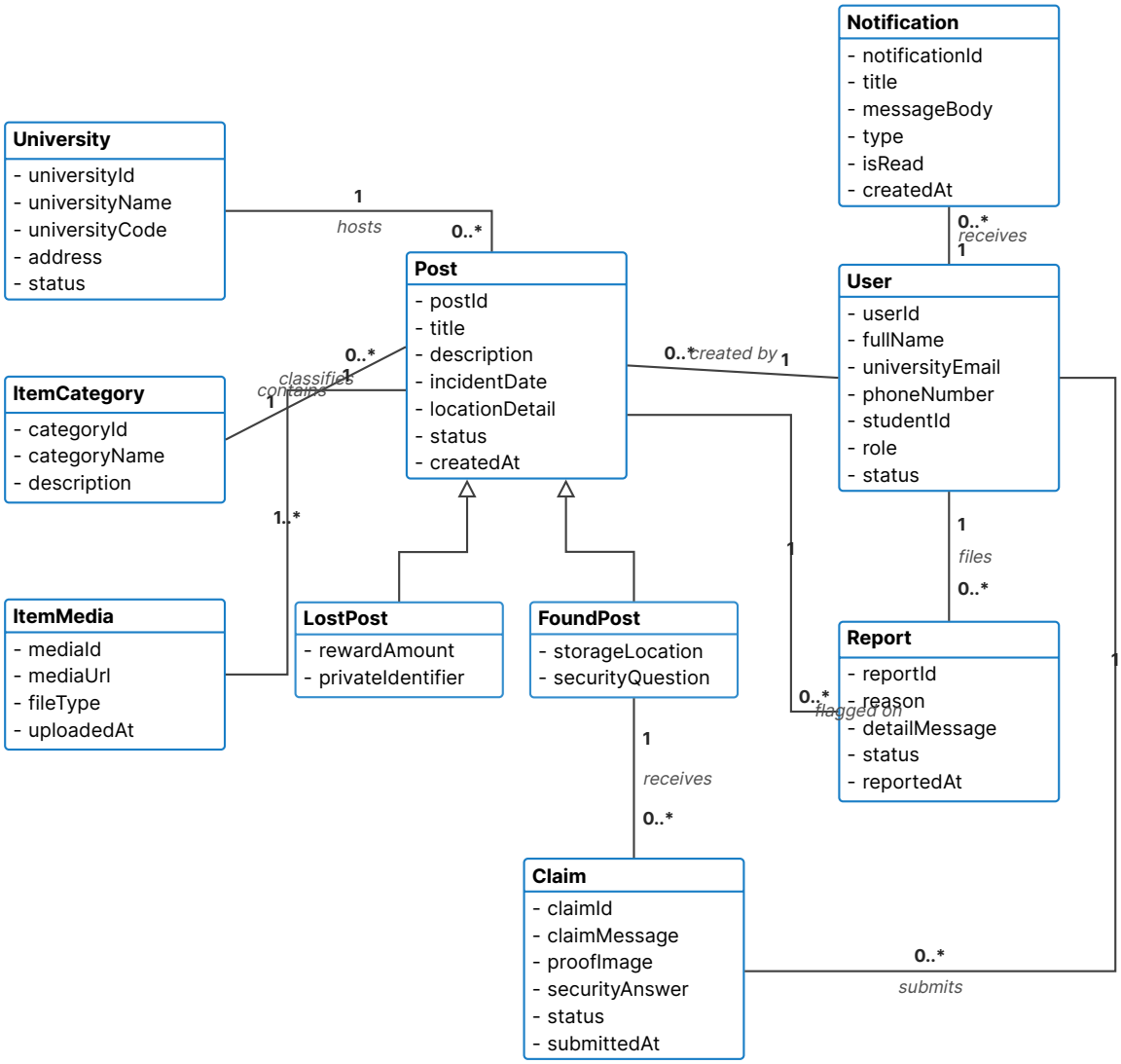


Figure 1: Domain Model diagram for US-Recovery

4.1.1 Domain Entities Specification

Bảng dưới đây đặc tả chi tiết ý nghĩa nghiệp vụ và các thuộc tính khái niệm của từng thực thể trong miền bài toán:

Thực thể	Định nghĩa nghiệp vụ	Thuộc tính khái niệm	Quy tắc ràng buộc nghiệp vụ

User	Đại diện cho người dùng đã định danh trong trường đại học hoặc quản trị viên hệ thống.	userId, fullName, universityEmail, phoneNumber, studentId, role (GUEST, STUDENT, ADMIN), status (ACTIVE, BANNED)	Bắt buộc phải có email đuôi @...edu.vn đã xác thực (BR-01). Số điện thoại được bảo vệ mặc định (BR-02).
University	Trường đại học nơi xảy ra sự việc mất hoặc nhặt được vật phẩm.	universityId, universityName, universityCode, address, status (APPROVED, PENDING)	Các trường/cơ sở mới do đại diện trường đề xuất cần được Quản trị viên xét duyệt trước khi hiển thị (FR-08).
ItemCategory	Danh mục phân loại vật phẩm (ví dụ: Thẻ sinh viên, Thiết bị điện tử, Ví/Bóp, Chìa khóa, Balo/Túi).	categoryId, categoryName, description	Do Quản trị viên quản lý tập trung; hỗ trợ phân loại cho bộ lọc tìm kiếm (FR-04).
Post	Thực thể trừu tượng đại diện cho một tin báo cáo vật phẩm thất lạc hoặc nhặt được trên khuôn viên trường.	postId, title, description, incidentDate, locationDetail, status (ACTIVE, PENDING_VERIFICATION, RESOLVED, CLOSED, REPORTED), createdAt	Mỗi bài đăng thuộc đúng một người tạo và gắn với một trường đại học; tự động chuyển sang hàng đợi nếu có ≥ 3 báo cáo vi phạm (BR-03).
LostPost	Chuyên biệt hóa của Post cho trường hợp sinh viên thông báo tài sản cá nhân bị thất lạc.	Kế thừa từ Post, bổ sung: rewardAmount (khoản hậu tạ tự nguyện), privateIdentifier (đặc điểm nhận dạng kín).	Chỉ chủ sở hữu bài đăng mới có thể cập nhật trạng thái bài đăng sang Resolved khi tìm lại được đồ.
FoundPost	Chuyên biệt hóa của Post cho trường hợp nhặt được vật phẩm và muốn tìm chủ sở hữu.	Kế thừa từ Post, bổ sung: storageLocation (nơi lưu giữ hiện tại: Bảo vệ/Tự giữ), securityQuestion (câu hỏi bảo mật).	Cung cấp câu hỏi bảo mật để người nhận (Claimant) chứng minh quyền sở hữu trước khi mở khóa thông tin liên hệ (BR-02).
ItemMedia	Hình ảnh thực tế của vật phẩm thất lạc hoặc khu vực xảy ra vụ việc.	mediaId, mediaUrl, fileType, uploadedAt	Mỗi bài đăng có thể đính kèm nhiều ảnh (1..*); ảnh thẻ sinh viên có thể che mờ thông tin riêng tư (FR-10).
Claim	Yêu cầu nhận lại vật phẩm do một sinh viên gửi tới một bài đăng nhặt đồ (FoundPost).	claimId, claimMessage, proofImage, securityAnswer, status (PENDING, ACCEPTED, REJECTED, CANCELLED), submittedAt	Người gửi Claim phải trả lời câu hỏi xác minh hoặc cung cấp ảnh chứng minh; người nhặt có quyền duyệt hoặc từ chối (FR-05, FR-06).

Report	Báo cáo phản ánh bài đăng có dấu hiệu vi phạm quy định (lừa đảo, thông tin sai sự thật, spam).	reportId, reason, detailMessage, status (PENDING, APPROVED, DISMISSED), reportedAt	Sinh viên chỉ được báo cáo 1 lần cho mỗi bài đăng; bài viết có ≥ 3 báo cáo tự động chuyển vào hàng đợi kiểm duyệt (BR-03).
Notification	Thông báo hệ thống gửi tới sinh viên nhằm cập nhật tiến trình nhận đồ hoặc tương tác tài khoản.	notificationId, title, messageBody, type (CLAIM_RECEIVED, CLAIM_DECIDED, POST_MODERATED), isRead, createdAt	Gửi tự động cho người dùng đích khi có sự kiện tương tác Claim hoặc thay đổi trạng thái bài đăng (FR-09).

4.1.2 Domain Relationships and Multiplicities

Các mối quan hệ cấu trúc và ngữ nghĩa giữa các thực thể trong mô hình lĩnh vực được tóm tắt trong bảng sau:

Thực thể nguồn	Thực thể đích	Tên quan hệ	Bội số	Ý nghĩa nghiệp vụ
University	Post	hosts / located_at	1 : 0..*	Một trường đại học có thể diễn ra nhiều bài đăng mất/nhặt đồ; mỗi bài đăng bắt buộc gắn với đúng 1 trường đại học.
ItemCategory	Post	classifies	1 : 0..*	Một danh mục chứa nhiều bài đăng; mỗi bài đăng được phân loại vào đúng 1 danh mục vật phẩm.
Post	ItemMedia	contains	1 : 1..*	Một bài đăng chứa ít nhất 1 ảnh minh họa thực tế; mỗi ảnh đính kèm thuộc về đúng 1 bài đăng.
User	Post	creates	1 : 0..*	Một người dùng có thể tạo nhiều bài đăng theo thời gian; mỗi bài đăng do đúng 1 người dùng sở hữu.
Post	LostPost / FoundPost	generalizes	1 : 1	Post là lớp cha trừu tượng khái niệm hóa các đặc tính chung của bài đăng mất đồ (LostPost) và nhặt đồ (FoundPost).

FoundPost	Claim	receives	1 : 0..*	Một bài đăng nhặt đồ có thể nhận nhiều yêu cầu nhận đồ từ nhiều sinh viên khác nhau; mỗi Claim gửi đến đúng 1 FoundPost.
User	Claim	submits	1 : 0..*	Một sinh viên có thể nộp nhiều yêu cầu Claim cho các tài sản thất lạc khác nhau; mỗi Claim do đúng 1 sinh viên gửi.
Post	Report	flagged_on	1 : 0..*	Một bài đăng có thể bị khiếu nại/báo cáo bởi nhiều người dùng khác nhau; mỗi Report trỏ tới đúng 1 bài đăng.
User	Report	files	1 : 0..*	Một người dùng có thể gửi báo cáo vi phạm đối với các bài đăng nghi vấn; mỗi Report do đúng 1 người gửi.
User	Notification	receives	1 : 0..*	Một người dùng nhận nhiều thông báo trong quá trình tương tác; mỗi thông báo thuộc quyền sở hữu của 1 người dùng.

4.1.3 Business Rules Verification in Domain Model

Bốn quy tắc nghiệp vụ then chốt từ PA2 được ánh xạ và xác minh trực tiếp trên các thực thể khái niệm của Domain Model:

1. **Verified School Email (BR-01):** Thực thể `User` quản lý thuộc tính `universityEmail` (bắt buộc đuôi tên miền giáo dục hợp lệ) và trạng thái `status = ACTIVE` sau khi hoàn tất xác thực OTP/link. Các mối quan hệ tạo bài (`creates`) và gửi yêu cầu nhận đồ (`submits`) chỉ hợp lệ đối với `User` đã qua kiểm tra chính danh này.
2. **Protected Contact Details (BR-02):** Thuộc tính liên hệ riêng tư (`phoneNumber`, email cá nhân) được mô hình hóa gắn liền với `User`, nhưng độc lập với thực thể `Post`. Không có liên kết trực tiếp nào bộc lộ thông tin liên hệ của chủ bài đăng cho các sinh viên khác; thông tin này chỉ được chia sẻ theo ngữ cảnh cho một `User` đóng vai trò Claimant khi thực thể `Claim` chuyển sang trạng thái `ACCEPTED`.
3. **Claim Approval Workflow (BR-04 / FR-05, FR-06):** Quan hệ giữa `FoundPost` và `Claim` cho phép

một bài đăng nhận nhiều yêu cầu đối chất. Trạng thái của thực thể `Claim` tuân thủ vòng đời nghiêm ngặt: `PENDING` → `ACCEPTED` / `REJECTED`. Khi một `Claim` được chấp thuận bởi chủ bài đăng, một liên kết trạng thái đồng thời kích hoạt chuyển `Post.status` sang `RESOLVED`, khép lại bài toán tìm đồ.

4. **Moderation Threshold (BR-03 / FR-07):** Quan hệ kết tập `Post` (1) -- (0..*) `Report` ghi nhận các khiếu nại từ cộng đồng. Bất biến nghiệp vụ quy định: nếu $\sum \text{Report} \geq 3$, bài đăng vi phạm sẽ tự động kích hoạt chuyển trạng thái `Post.status = REPORTED`, tạm thời thu hồi quyền hiển thị công khai trên thanh tìm kiếm và đưa vào danh sách kiểm duyệt ưu tiên của Quản trị viên.

4.2 Class Diagram

[Design-level class diagram: class names, relationships (type, multiplicity). Include attributes and operations if the diagram is not too large; split by package if needed.]

4.3 Class Specifications

[Specify the 6--8 most important classes.]

4.3.1 Class C1

[Superclass (if any), responsibility of the class.]

#	Attribute	Type	Visibility	Constraint	Description

#	Operation	Parameters / return	Visibility	Description

4.4 Sequence Diagrams

[Draw a sequence diagram for the main flow of 3--5 key use cases (include at least one alt fragment). Every message should correspond to an operation in the class diagram.]

4.5 State Machine Diagrams

[For objects whose behaviour depends on their status (order, booking, account, ...), draw a state machine diagram.]

4.6 Design Principles and Patterns

Principle / pattern	Where it is applied	Why
<e.g., Strategy>	<ticket pricing>	<different discount rules>

5 Data Design

5.1 Data Diagram

[ERD / database diagram of the stored data and their relationships (MySQL Workbench, dbdiagram.io, ...).]

5.2 Data Specification

[For a DBMS: describe each table. For JSON/XML/file storage: describe the structure, data types, constraints and give a sample file.]

Table	Column	Data type	Constraints (PK, FK, NOT NULL, UNIQUE, CHECK)	Description

5.3 Normalisation and Indexes

[State the normal form reached (at least 3NF, or justify denormalisation) and the indexes created for frequent queries.]

6 User Interface and User Experience Design

6.1 Screen List and Screen Flow

[Draw the screen flow diagram (navigation between screens).]

#	Screen	Description	Related use case

6.2 Screen Specifications

[Specify the 5--6 most important screens: design image, controls, events and validation. For the other screens, the design image is enough.]

6.2.1 Screen ``A''

#	Control	Type	Description / validation	Event handling

6.3 Accessibility Checklist

- Text contrast $\geq 4.5{:}1$; colour is not the only way to convey information
- All functions usable with the keyboard; visible focus
- Form fields have labels; error messages explain how to fix the problem
- Layout works on mobile and desktop (responsive)

7 Traceability Update

[Extend the traceability matrix from PA2 with the design elements (component, class, screen) that realise each requirement.]

8 Changes to the Requirements Since PA2

[List any changes made to the PA2 requirements after submission: what changed, why, and which items were affected.]

#	Change	Reason	Affected items

9 AI Usage Log

[Required in every PA (course GenAI policy). List each significant use of AI tools. Write ``No AI tools were used'' if none. Undisclosed AI use is treated as an academic integrity violation.]

#	Member	Tool	Task / purpose	How the output was verified and changed
1		<e.g., ChatGPT, Claude, Copilot>	<e.g., draft use case UC-03 specification>	<e.g., reviewed with the BA, corrected 2 alternative flows>